

of the measured signal C into signal and noise components S and N can usefully be done “by eye” from a crude plot of power spectral density. All of this may give you thoughts about iterating the procedure we have just described. For example, after designing a filter with response $\Phi(f)$ and using it to make a respectable guess at the signal $\tilde{U}(f) = \Phi(f)C(f)/R(f)$, you might turn about and regard $\tilde{U}(f)$ as a fresh new signal that you could improve even further with the same filtering technique. Don’t waste your time on this line of thought. The scheme converges to a signal of $S(f) = 0$. Converging iterative methods do exist; this just isn’t one of them.

You can use the routine `four1` (§12.2) or `realft` (§12.3) to FFT your data when you are constructing an optimal filter. To apply the filter to your data, you can use the methods described in §13.1. The specific routine `conv1v` is not needed for optimal filtering, since your filter is constructed in the frequency domain to begin with. If you are also deconvolving your data with a known response function, however, you can modify `conv1v` to multiply by your optimal filter just before it takes the inverse Fourier transform.

CITED REFERENCES AND FURTHER READING:

- Rabiner, L.R., and Gold, B. 1975, *Theory and Application of Digital Signal Processing* (Englewood Cliffs, NJ: Prentice-Hall).
- Nussbaumer, H.J. 1982, *Fast Fourier Transform and Convolution Algorithms* (New York: Springer).
- Elliott, D.F., and Rao, K.R. 1982, *Fast Transforms: Algorithms, Analyses, Applications* (New York: Academic Press).

13.4 Power Spectrum Estimation Using the FFT

In the previous section we “informally” estimated the power spectral density of a function $c(t)$ by taking the modulus-squared of the discrete Fourier transform of some finite, sampled stretch of it. In this section we’ll do roughly the same thing, but with considerably greater attention to details. This attention will uncover some surprises.

The first detail is the normalization of the power spectrum (or *power spectral density* or *PSD*). In general, there is *some* relation of proportionality between a measure of the squared amplitude of the function and a measure of the amplitude of the PSD. Unfortunately, there are several different conventions for describing the normalization in each domain, and many opportunities for getting wrong the relationship between the two domains. Suppose that our function $c(t)$ is sampled at N points to produce values $c_0 \dots c_{N-1}$, and that these points span a range of time T , that is, $T = (N - 1)\Delta$, where Δ is the sampling interval. Then here are several different descriptions of the total power:

$$\sum_{j=0}^{N-1} |c_j|^2 \equiv \text{sum squared amplitude} \quad (13.4.1)$$

$$\frac{1}{T} \int_0^T |c(t)|^2 dt \approx \frac{1}{N} \sum_{j=0}^{N-1} |c_j|^2 \equiv \text{mean squared amplitude} \quad (13.4.2)$$

$$\int_0^T |c(t)|^2 dt \approx \Delta \sum_{j=0}^{N-1} |c_j|^2 \equiv \text{time-integral squared amplitude} \quad (13.4.3)$$

PSD estimators, as we shall see, have an even greater variety. In this section, we consider a class of them that give estimates at discrete values of frequency f_i , where i will range over integer values. In the next section, we will learn about a different class of estimators that produce estimates that are continuous functions of frequency f . Even if it is agreed always to relate the PSD normalization to a particular description of the function normalization (e.g., 13.4.2), there are at least the following possibilities: The PSD is

- defined for discrete positive, zero, and negative frequencies, and its sum over these is the function mean squared amplitude
- defined for zero and discrete positive frequencies only, and its sum over these is the function mean squared amplitude
- defined in the Nyquist interval from $-f_c$ to f_c , where $f_c = 1/(2\Delta)$, and its integral over this range is the function mean squared amplitude
- defined from 0 to f_c , and its integral over this range is the function mean squared amplitude (it *never* makes sense to integrate the PSD of a sampled function outside of the Nyquist interval $-f_c$ and f_c since, according to the sampling theorem, power there will have been aliased into the Nyquist interval).

It is hopeless to define enough notation to distinguish all possible combinations of normalizations. In what follows, we use the notation $P(f)$ to mean *any* of the above PSDs, stating in each instance how the particular $P(f)$ is normalized. Beware the inconsistent notation in the literature.

The method of power spectrum estimation used in the previous section is a simple version of an estimator called, historically, the *periodogram*. If we take an N -point sample of the function $c(t)$ at equal intervals and use the FFT to compute its discrete Fourier transform

$$C_k = \sum_{j=0}^{N-1} c_j e^{2\pi i j k / N} \quad k = 0, \dots, N-1 \quad (13.4.4)$$

then the periodogram estimate of the power spectrum is defined at $N/2 + 1$ frequencies as

$$\begin{aligned} P(0) &= P(f_0) = \frac{1}{N^2} |C_0|^2 \\ P(f_k) &= \frac{1}{N^2} [|C_k|^2 + |C_{N-k}|^2] \quad k = 1, 2, \dots, \left(\frac{N}{2} - 1\right) \\ P(f_c) &= P(f_{N/2}) = \frac{1}{N^2} |C_{N/2}|^2 \end{aligned} \quad (13.4.5)$$

where f_k is defined only for the zero and positive frequencies

$$f_k \equiv \frac{k}{N\Delta} = 2f_c \frac{k}{N} \quad k = 0, 1, \dots, \frac{N}{2} \quad (13.4.6)$$

By Parseval's theorem, equation (12.1.10), we see immediately that equation (13.4.5) is normalized so that the sum of the $N/2 + 1$ values of P is equal to the mean squared amplitude of the function c_j .

Here is an object (Spectreg for "spectrum register") that implements equations (13.4.4) – (13.4.6). Its constructor takes an integer argument M that defines both the number of data points, $2M \equiv N$, and $M + 1$, the number of frequencies in the estimate between 0 and f_c , inclusive. M must be a power of 2. Spectreg has some other features that we will build on below when we learn about window functions and variance reduction. For now, the function window that appears should be defined as returning the constant 1, that is,

```
Doub window(Int j,Int n) {return 1.;}

```

spectrum.h

```
struct Spectreg {
Object for accumulating power spectrum estimates from one or more segments of data.
    Int m,m2,nsum;
    VecDoub specsum, wksp;

    Spectreg(Int em) : m(em), m2(2*m), nsum(0), specsum(m+1,0.), wksp(m2) {
    Constructor. Sets  $M$ , such that data segments will have length  $2M$ , and the spectrum will
    be estimated at  $M + 1$  frequencies.
        if (m & (m-1)) throw("m must be power of 2");
    }

    template<class D>
    void adddataseg(VecDoub_I &data, D &window) {
    Process a data segment of length  $2M$  using the window function, which can be either a
    bare function or a functor.
        Int i;
        Doub w,fac,sumw = 0.;
        if (data.size() != m2) throw("wrong size data segment");
        for (i=0;i<m2;i++) {
            w = window(i,m2);
            wksp[i] = w*data[i];
            sumw += SQR(w);
        }
        fac = 2./(sumw*m2);
        realft(wksp,1);
        specsum[0] += 0.5*fac*SQR(wksp[0]);
        for (i=1;i<m;i++) specsum[i] += fac*(SQR(wksp[2*i])+SQR(wksp[2*i+1]));
        specsum[m] += 0.5*fac*SQR(wksp[1]);
        nsum++;
    }

    VecDoub spectrum() {
    Return power spectrum estimates as a vector. You can instead just access specsum directly,
    and divide by nsum.
        VecDoub spec(m+1);
        if (nsum == 0) throw("no data yet");
        for (Int i=0;i<=m;i++) spec[i] = specsum[i]/nsum;
        return spec;
    }

    VecDoub frequencies() {
    Return vector of frequencies (in units of  $1/\Delta$ ) at which estimates are made.
        VecDoub freq(m+1);
        for (Int i=0;i<=m;i++) freq[i] = i*0.5/m;
        return freq;
    }
};

```

The naive use of Spectreg would be as follows: Declare an instance with a power-of-two value of M . Call `adddataseg` to process a vector of data, length $2M$. Call `spectrum` and `frequencies` to get, respectively, the PSD estimates and the frequencies at which they are made (in units of $1/\Delta$).

Before we rush to use Spectreg, however, we must now ask this question: In what sense is the periodogram estimate (13.4.5) a “true” estimator of the power spectrum of the underlying function $c(t)$? You can find the answer treated in considerable detail in the literature cited (see, e.g., [1] for an introduction). Here is a summary.

First, is the *expectation value* of the periodogram estimate equal to the power spectrum, i.e., is the estimator correct (unbiased) on average? Well, yes and no. We wouldn’t really expect one of the $P(f_k)$ ’s to equal the continuous $P(f)$ at *exactly* f_k , since f_k is supposed to be representative of a whole frequency “bin” extending from halfway from the preceding discrete frequency to halfway to the next one. We *should* be expecting the $P(f_k)$ to be some kind of average of $P(f)$ over a narrow window function centered on its f_k . For the periodogram estimate (13.4.6), that window function, as a function of s the frequency offset *in bins*, is

$$W(s) = \frac{1}{N^2} \left[\frac{\sin(\pi s)}{\sin(\pi s/N)} \right]^2 \quad (13.4.7)$$

Notice that $W(s)$ has oscillatory lobes but, apart from these, falls off only about as $W(s) \approx (\pi s)^{-2}$. This is not a very rapid fall-off, and it results in significant *leakage* (that is the technical term) from one frequency to another in the periodogram estimate. Notice also that $W(s)$ happens to be zero for s equal to a nonzero integer. This means that if the function $c(t)$ is a pure sine wave of frequency exactly equal to one of the f_k ’s, then there will be *no* leakage to adjacent f_k ’s. But this is not the characteristic case! If the frequency is, say, one-third of the way between two adjacent f_k ’s, then the leakage will extend *well* beyond those two adjacent bins. The solution to the problem of leakage is called *data windowing*, and we will discuss it below.

Turn now to another question about the periodogram estimate. What is the variance of that estimate as N goes to infinity? In other words, as we take more sampled points from the original function (either sampling a longer stretch of data at the same sampling rate, or else by resampling the same stretch of data with a faster sampling rate), then how much more accurate do the estimates P_k become? The unpleasant answer is that the periodogram estimates *do not become more accurate at all!* In fact, the variance of the periodogram estimate at a frequency f_k is always equal to the square of its expectation value at that frequency. In other words, the standard deviation is always 100% of the value, independent of N !

How can this be? Where did all the information go as we added points? It all went into producing estimates at a greater number of discrete frequencies f_k . If we sample a longer run of data using the same sampling rate, then the Nyquist critical frequency f_c is unchanged, but we now have finer frequency resolution (more f_k ’s) within the Nyquist frequency interval; alternatively, if we sample the same length of data with a finer sampling interval, then our frequency resolution is unchanged, but the Nyquist range now extends up to a higher frequency. In neither case do the additional samples reduce the variance of any one particular frequency’s estimated PSD.

You don’t have to live with PSD estimates with 100% standard deviations, however. You simply have to know some techniques for reducing the variance of the

estimates. Here are two techniques that are very nearly identical mathematically, though different in implementation. The first is to compute a periodogram estimate with finer discrete frequency spacing than you really need, and then to sum the periodogram estimates at K consecutive discrete frequencies to get one “smoother” estimate at the mid-frequency of those K . The variance of that summed estimate will be smaller than the estimate itself by a factor of exactly $1/K$, i.e., the standard deviation will be smaller than 100% by a factor $1/\sqrt{K}$. Thus, to estimate the power spectrum at $M + 1$ discrete frequencies between 0 and f_c inclusive, you begin by taking the FFT of $2MK$ points (which number had better be an integer power of 2!). You then take the modulus square of the resulting coefficients, add positive and negative frequency pairs, and divide by $(2MK)^2$, all according to equation (13.4.5) with $N = 2MK$. Finally, you “bin” the results into summed (not averaged) groups of K . This procedure is very easy to program, so we will not bother to give a routine for it. The reason that you sum, rather than average, K consecutive points is so that your final PSD estimate will preserve the normalization property that the sum of its $M + 1$ values equals the mean square value of the function.

A second technique for estimating the PSD at $M + 1$ discrete frequencies in the range 0 to f_c is to partition the original sampled data into K segments each of $2M$ consecutive sampled points. Each segment is separately FFT'd to produce a periodogram estimate (equation 13.4.5 with $N \equiv 2M$). Finally, the K periodogram estimates are averaged at each frequency. It is this final averaging that reduces the variance of the estimate by a factor K (standard deviation by $\sqrt{K}$). This second technique is computationally more efficient than the first technique above by a modest factor, since it is logarithmically more efficient to take many shorter FFTs than one longer one. The principal advantage of the second technique, however, is that only $2M$ data points are manipulated at a single time, not $2KM$ as in the first technique. This means that the second technique is the natural choice for processing long runs of data, as from a real-time device or slow storage.

In fact, you may already have noticed, the object `Spectreg` implements this second technique. If you call `adddataseg` K times, with a different vector of $2M$ data points each time, then the result returned by `spectrum` is the average of the K periodograms. However, we should *still* not rush to use `Spectreg`. We need first to return to the matters of leakage and data windowing that were brought up after equation (13.4.7) above.

13.4.1 Data Windowing

The purpose of data windowing is to modify equation (13.4.7), which expresses the relation between the spectral estimate P_k at a discrete frequency and the actual underlying continuous spectrum $P(f)$ at nearby frequencies. In general, the spectral power in one “bin” k contains leakage from frequency components that are actually s bins away, where s is the independent variable in equation (13.4.7). There is, as we pointed out, quite substantial leakage even from moderately large values of s . Note that s is not an integer, in general, because actual frequencies can have any real value.

When we select a run of N sampled points for periodogram spectral estimation, we are in effect multiplying an infinite run of sampled data c_j by a window function in time, one that is zero except during the total sampling time $N\Delta$ and is unity during that time. In other words, the data are windowed by a square window function. By the convolution theorem (12.0.10; but interchanging the roles of f and t), the Fourier

transform of the product of the data with this square window function is equal to the convolution of the data's Fourier transform with the window's Fourier transform. In fact, equation (13.4.7) is nothing more than the square of the discrete Fourier transform of the unity window function.

$$W(s) = \frac{1}{N^2} \left[\frac{\sin(\pi s)}{\sin(\pi s/N)} \right]^2 = \frac{1}{N^2} \left| \sum_{k=0}^{N-1} e^{2\pi i s k/N} \right|^2 \quad (13.4.8)$$

The reason for the leakage at large values of s is that the square window function turns on and off so rapidly. Its Fourier transform has substantial components at high frequencies. To remedy this situation, we can multiply the input data c_j , $j = 0, \dots, N-1$ by a window function w_j that changes more gradually from zero to a maximum and then back to zero as j ranges from 0 to N . In this case, the equations for the periodogram estimator (13.4.4 – 13.4.5) become

$$D_k \equiv \sum_{j=0}^{N-1} c_j w_j e^{2\pi i j k/N} \quad k = 0, \dots, N-1 \quad (13.4.9)$$

$$\begin{aligned} P(0) &= P(f_0) = \frac{1}{W_{ss}} |D_0|^2 \\ P(f_k) &= \frac{1}{W_{ss}} [|D_k|^2 + |D_{N-k}|^2] \quad k = 1, 2, \dots, \left(\frac{N}{2} - 1\right) \\ P(f_c) &= P(f_{N/2}) = \frac{1}{W_{ss}} |D_{N/2}|^2 \end{aligned} \quad (13.4.10)$$

where W_{ss} stands for “window squared and summed,”

$$W_{ss} \equiv N \sum_{j=0}^{N-1} w_j^2 \quad (13.4.11)$$

and f_k is given by (13.4.6). The more general form of (13.4.7) can now be written in terms of the window function w_j as

$$W(s) = \frac{1}{W_{ss}} \left| \sum_{k=0}^{N-1} e^{2\pi i s k/N} w_k \right|^2 \quad (13.4.12)$$

There is a lot of perhaps unnecessary lore about the choice of a window function, and practically every function that rises from zero to a peak and then falls again has been named after someone. A few of the more common (also shown in Figure 13.4.1) are

$$w_j = 1 - \left| \frac{j - \frac{1}{2}N}{\frac{1}{2}N} \right| \equiv \text{Bartlett window} \quad (13.4.13)$$

(the “Parzen window” is a smoother, but similarly shaped, functional form)

$$w_j = \frac{1}{2} \left[1 - \cos \left(\frac{2\pi j}{N} \right) \right] \equiv \text{Hann window} \quad (13.4.14)$$

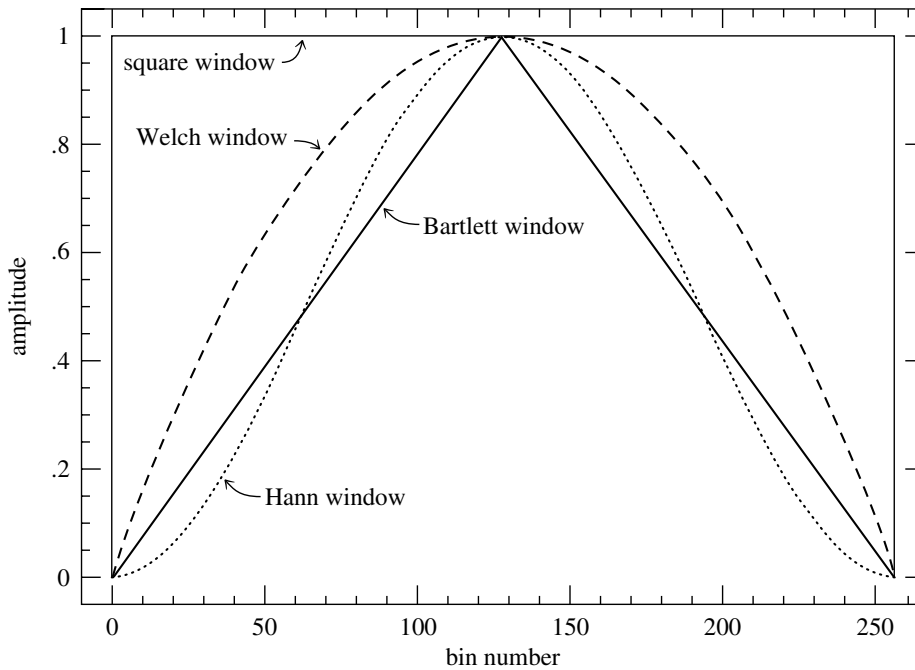


Figure 13.4.1. Window functions commonly used in FFT power spectral estimation. The data segment, here of length 256, is multiplied (bin by bin) by the window function before the FFT is computed. The square window, which is equivalent to no windowing, is least recommended. The Welch and Bartlett windows are good choices.

(the “Hamming window” is similar but does not go exactly to zero at the ends)

$$w_j = 1 - \left(\frac{j - \frac{1}{2}N}{\frac{1}{2}N} \right)^2 \equiv \text{Welch window} \quad (13.4.15)$$

We are inclined to follow Welch in recommending that you use either (13.4.13) or (13.4.15) in practical work. However, at the level of the discussion thus far, there is *little difference* between any of these (or similar) window functions. Their difference lies in subtle trade-offs among the various figures of merit that can be used to describe the narrowness or peakedness of the spectral leakage functions computed by (13.4.12). These figures of merit have such names as: *highest sidelobe level (dB)*, *sidelobe fall-off (dB per octave)*, *equivalent noise bandwidth (bins)*, *3-dB bandwidth (bins)*, *scallop loss (dB)*, and *worst-case process loss (dB)*. Roughly speaking, the principal trade-off is between making the central peak as narrow as possible versus making the tails of the distribution fall off as rapidly as possible. For details, see, e.g., [2]. Figure 13.4.2 plots the leakage amplitudes for several windows already discussed.

There is a particular lore about window functions that rise smoothly from zero to unity in the first small fraction (say 10%) of the data, then stay at unity until the last small fraction (again say 10%) of the data, during which the window function falls smoothly back to zero. These windows will squeeze a little bit of extra narrowness out of the main lobe of the leakage function (never as much as a factor of two,

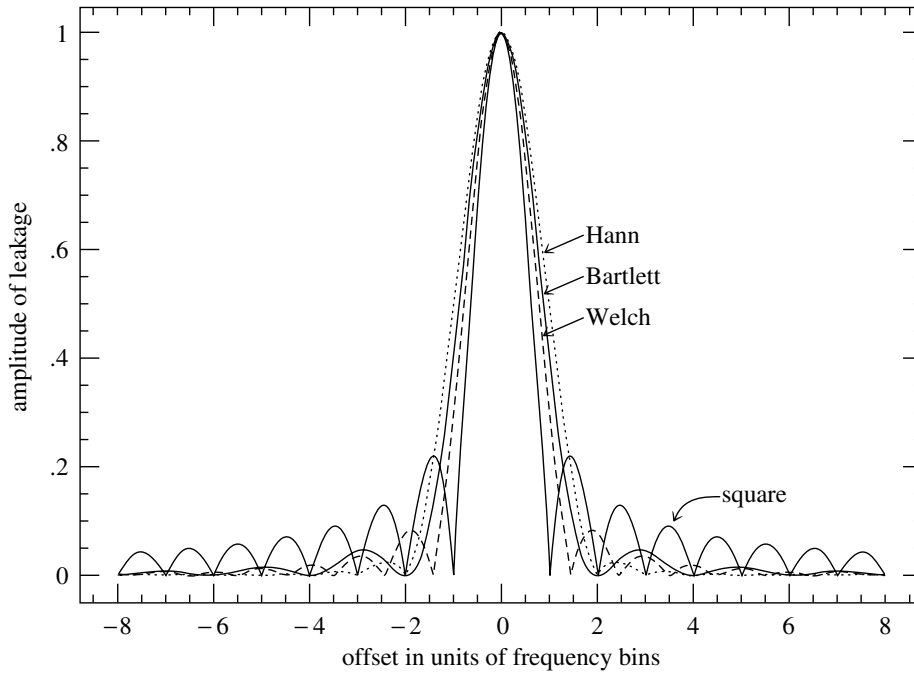


Figure 13.4.2. Leakage functions for the window functions of Figure 13.4.1. A signal whose frequency is actually located at zero offset “leaks” into neighboring bins with the amplitude shown. The purpose of windowing is to reduce the leakage at large offsets, where square (no) windowing has large sidelobes. Offset can have a fractional value, since the actual signal frequency can be located between two frequency bins of the FFT.

however), but trade this off by widening the leakage tail by a significant factor (e.g., the reciprocal of 10%, a factor of ten). If we distinguish between the *width* of a window (number of samples for which it is at its maximum value) and its *rise/fall time* (number of samples during which it rises and falls); and if we distinguish between the *FWHM* (full width to half maximum value) of the leakage function’s main lobe and the *leakage width* (full width that contains half of the spectral power that is not contained in the main lobe), then these quantities are related roughly by

$$(\text{FWHM in bins}) \approx \frac{N}{(\text{window width})} \quad (13.4.16)$$

$$(\text{leakage width in bins}) \approx \frac{N}{(\text{window rise/fall time})} \quad (13.4.17)$$

For the windows given above in (13.4.13) – (13.4.15), the effective window widths and the effective window rise/fall times are both of order $\frac{1}{2}N$. Generally speaking, we feel that the advantages of windows whose rise and fall times are only small fractions of the data length are minor or nonexistent, and we avoid using them. One sometimes hears it said that flat-topped windows “throw away less of the data,” but we will show you two better ways of dealing with that problem, namely by overlapping data segments or by multitaper methods.

Now, at last, we really *are* ready to use the `Spectreg` object. First, choose a window function. The templating in `Spectreg` allows it to accept either a bare

function or a functor. Use the former if your window function is fast to compute, or the latter if you want to precompute and store a more complicated window function, or one with auxiliary parameters. Examples are

spectrum.h	<pre>Doub square(Int j, Int n) {return 1.;} Doub bartlett(Int j, Int n) {return 1.-abs(2.*j/(n-1.)-1.);} Doub welch(Int j, Int n) {return 1.-SQRT(2.*j/(n-1.)-1.);}</pre>	Don't use this! Use this, or this...
<pre>struct Hann { ...or this. This is an example of a functor. Int nn; VecDoub win; Hann(Int n) : nn(n), win(n) { Constructor. Compute and store the window function in a table. Doub twopi = 8.*atan(1.); for (Int i=0;i<nn;i++) win[i] = 0.5*(1.-cos(i*twopi/(nn-1.))); } Doub operator() (Int j, Int n) { Make it a functor, able to return values as if it were a function. if (n != nn) throw("incorrect n for this Hann"); return win[j]; } };</pre>		

Second, pick a value of M and declare a Spectreg object. Third, process K data segments, each of length $2M$. The larger K , the more accurate your answer. Fourth, get the PSD estimate at $M + 1$ frequencies by a call to the spectrum method (or, to avoid copying vectors, directly from the specsum member vector).

13.4.2 Overlapping Data Segments

We introduced window functions to mitigate leakage, a major problem, but in doing so we have created a new problem, luckily minor, which we now address. All good window functions, because they approach zero at their endpoints, deweight, and in effect throw away, valid data. A consequence is that, for any number of data segments K each of length $2M$, the variance of the PSD estimate is somewhat larger with a good window function than with the (bad) square window.

Sometimes you are not limited in the number of data points, but rather by the computer resources to process them. For example, the data may be pouring out of a real-time device at a high rate. In such a situation, data deweighting is not an issue. You should use Spectreg as already described, accumulating as many data segments as you need to obtain the desired accuracy. Indeed, this gives you the smallest variance estimate per computer operation.

More often, however, you are limited in the total number of data points, and you want to get the smallest variance estimate from them, but without giving up the low-leakage benefit of windowing. In this situation it turns out to be optimal, or nearly optimal, to overlap the segments by one-half of their length. The first and second sets of M points become segment number 1 (length $2M$ as usual); the second and third sets of M points become segment number 2; and so on, up to segment number K , which is made of the K th and $K + 1$ st sets of M points. The total number of sampled points is therefore $(K + 1)M$, just over half as many as with nonoverlapping segments. The reduction in the variance is not a full factor

of K , since the segments are not statistically independent. It can be shown that the variance is instead reduced by a factor of about $9K/11$ [3]. This is, however, significantly better than the reduction of about $K/2$ that would have resulted if the same *number* of data points were segmented without overlapping.

Here is an object `Spectolap`, derived from `Spectreg` as a base class, that implements the overlap method. As far as the user is concerned, the only difference is that the `adddataseg` method now requires a data segment of length M , not $2M$.

```
struct Spectolap : Spectreg {
```

[spectrum.h](#)

Object for power spectral estimation using overlapping data segments. The user sends non-overlapping segments of length M , which are processed in pairs of length $2M$, with overlap.

```
    Int first;
    VecDoub fullseg;

    Spectolap(Int em) : Spectreg(em), first(1), fullseg(2*em) {}
    Constructor. Sets  $M$ .

    template<class D>
    void adddataseg(VecDoub_I &data, D &window) {
    Process a data segment of length  $M$  using the window function, which can be either a bare
    function or a functor.
        Int i;
        if (data.size() != m) throw("wrong size data segment");
        if (first) {
            for (i=0;i<m;i++) fullseg[i+m] = data [i];
            first = 0;
            First segment is just stored.
        } else {
            Subsequent segments are processed.
            for (i=0;i<m;i++) {
                fullseg[i] = fullseg[i+m];
                fullseg[i+m] = data [i];
            }
            Spectreg::adddataseg(fullseg,window);
            Base class method, the data length
            is  $2M$ .
        }
    }

    template<class D>
    void addlongdata(VecDoub_I &data, D &window) {
    Process a long vector of data as overlapping segments each of length  $2M$ .
        Int i, k, noff, nt=data.size(), nk=(nt-1)/m;
        Doub del = nk > 1 ? (nt-m2)/(nk-1.) : 0.;
        Target separation.
        if (nt < m2) throw("data length too short");
        for (k=0;k<nk;k++) {
            Process nk overlapping segments.
            noff = (Int)(k*del+0.5);
            Offset is nearest integer.
            for (i=0;i<m2;i++) fullseg[i] = data[noff+i];
            Spectreg::adddataseg(fullseg,window);
        }
    }
};
```

The method `addlongdata` in `Spectolap` is provided to deal with another common situation: You want to estimate the PSD at $M + 1$ frequencies (as usual), but your data are in a long vector that is not necessarily a multiple of M , or $2M$, or a power of 2. Here we are assuming that the length of your data vector, N_{tot} , is much larger than $2M$. The problem is not that the number of segments K is small, but rather that K is not an integer. Overlapping data segments provide a nifty fix: We start with the next-larger integer number of segments, and then squeeze them together just a bit, like an accordion, until they exactly fit into N_{tot} . In other words,

we overlap them by slightly *more* than half of their length, to get an exact fit.

Here is our plain-vanilla recommendation for PSD estimation when your N_{tot} data points are not taxing the size of memory: Pick M , a power of 2, such that estimates at $M + 1$ frequencies between 0 and f_c (inclusive) are enough. Don't be too greedy on M , because the fractional standard deviation of your estimates will on the order of $(M/N_{\text{tot}})^{1/2}$. Then do

```
Int ntot=..., m=...;
VecDoub data(ntot), psd(m), freq(m);
...
Spectolap myspec(m);
myspec.addlongdata(data,bartlett);
psd = myspec.spectrum()
freq = myspec.frequencies()
```

13.4.3 Multitaper Methods and Slepian Functions

Multitaper methods provide a principled approach to the trade-off between (very) low leakage and minimizing the variance of the PSD estimate. If the leakage profiles in Figure 13.4.2 are acceptable to you (and see also Figure 13.4.4 below), then you don't need to read this section. In some applications, however, minimizing leakage is the whole game. For example, you may be looking for very weak spectral signals, either line or continuum, that can be masked by leakage from nearby strong lines. Or, you may be interested in the tail of a spectrum at high frequencies, which can be spuriously dominated by leakage from lower frequencies.

You have to give something to get something. Here, you have to accept a (small) broadening of the main lobe of the leakage function $W(s)$ in order to (greatly) suppress leakage outside of the main lobe. Broadening the main lobe is equivalent to giving up some frequency resolution. We can parameterize this by a value j_{res} . The goal is to minimize leakage for $|s| > j_{\text{res}}$, measured in bins, in exchange for which we are willing to have leakage near unity for any $|s| < j_{\text{res}}$. Typical values of j_{res} might be in the range of 2 to 10. (We'll see that larger values are not necessary.)

There are two key ideas in *multitaper methods*, somewhat independent of each other, originating in the work of Slepian [4]. The first idea is that, for a given data length N and choice j_{res} , one can actually solve for the *best possible* weights w_j , meaning the ones that make the leakage smallest among all possible choices. The beautiful and nonobvious answer (see [5]) is that the vector of optimal weights is the eigenvector corresponding to the smallest eigenvalue of the symmetric tridiagonal matrix with diagonal elements

$$\frac{1}{4} \left[N^2 - (N-1-2j)^2 \cos\left(\frac{2\pi j_{\text{res}}}{N}\right) \right], \quad j = 0, \dots, N-1 \quad (13.4.18)$$

and off-diagonal elements

$$-\frac{1}{2}j(N-j), \quad j = 1, \dots, N-1 \quad (13.4.19)$$

The second key idea is that the next few eigenvectors of this same matrix are also pretty good window functions. And because they are orthogonal to the first eigenvector (and each other), they give statistically independent estimates, which can be averaged together to decrease the variance of the final answer. Let k_T (for “taper”) denote the number of such estimates that are averaged. Figure 13.4.3 shows the first five window functions (eigenvectors number $k = 0, \dots, k_T - 1$) for the case $j_{\text{res}} = 3$, $N = 1024$. The functions (actually discrete sequences) obtained as eigenvectors of equations (13.4.18) and (13.4.19) are called *Slepian functions* or *discrete prolate spheroidal sequences* (*dpss*). You can see that larger values of k pick up the information in data regions that were deweighted in the first eigenvector $k = 0$. (You may have thought that window functions needed to be positive, but there is actually no such restriction in any of the discussion above.)

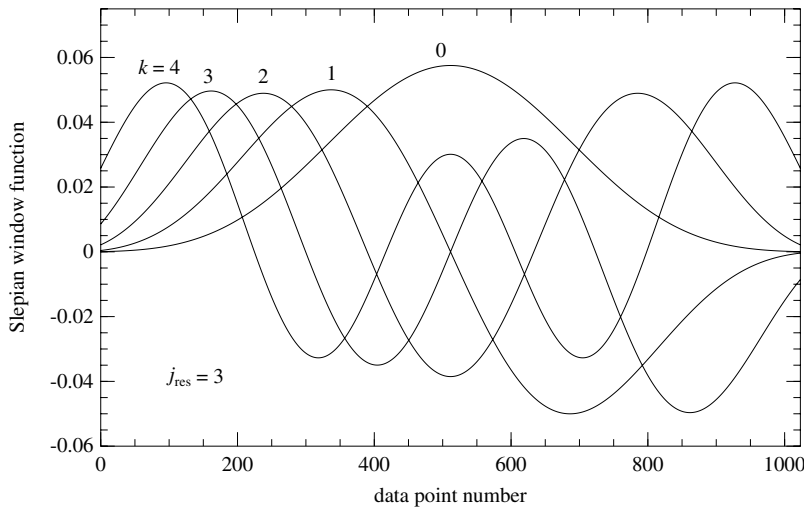


Figure 13.4.3. Slepian taper (window) functions for the case $j_{\text{res}} = 3$, $k = 0, 1, 2, 3, 4$, $N = 1024$. Combining the power spectrum estimates of different k 's uses, effectively, more of the data segment and decreases the variance.

The reason that you can't continue this indefinitely, using eigenvectors corresponding to larger and larger eigenvalues (increasing k_T), is that the leakage of the k th window function increases rather rapidly with k . Only $k_T < 2j_{\text{res}}$ values are worth considering at all, and only for $k_T \lesssim j_{\text{res}}$ is the leakage really tiny, which was, after all, the whole point. In Figure 13.4.3 you can already guess that $k = 3$ and 4 are going to have rather poor leakage properties, because they noticeably don't go to zero at their endpoints. Figure 13.4.4 shows the leakage function $W(s)$ for a variety of window functions, including those we met previously, now plotted on a logarithmic scale. Window functions shown as shaded have leakage so large as to be ruled out almost categorically; the square window is notable in this group. You can see how the main lobe of the Slepian functions extends almost exactly out to j_{res} , and that the suppression of the sidelobes of the lowest eigenvectors (e.g., Slepian 3,0 and 3,1) is really quite remarkable.

Here is an object, again derived from `Spectreg` as a base class, for estimating the PSD using the multitaper method with Slepian window functions. As in the base class, the method `adddataseg` accepts data segments of length $2M$, but it now adds to the average the result of the first k_T tapers of resolution j_{res} . Values for M , j_{res} and k_T are set in the constructor.

```
struct Slepian : Spectreg {
    Int jres, kt;
    MatDoub dpss;
    Doub p, pp, d, dd;
    Slepian(Int em, Int jjres, Int kkt)
    : Spectreg(em), jres(jjres), kt(kkt), dpss(kkt, 2*em) {
        Constructor sets  $M$  (same meaning as previously),  $j_{\text{res}}$ , and  $k_T$ , see text.
        if (jres < 1 || kt >= 2*jres) throw("kt too big or jres too small");
        filltable();
    }
    void filltable();
    void renorm(Int n) {
        Utility used by filltable.
        p = ldexp(p, n); pp = ldexp(pp, n); d = ldexp(d, n); dd = ldexp(dd, n);
    }
    struct Slepwindow {
        Captive function will be sent to the base class as a window function.
        Int k;
```

Table of Slepian.

Implementation in next subsection.

[spectrum.h](#)

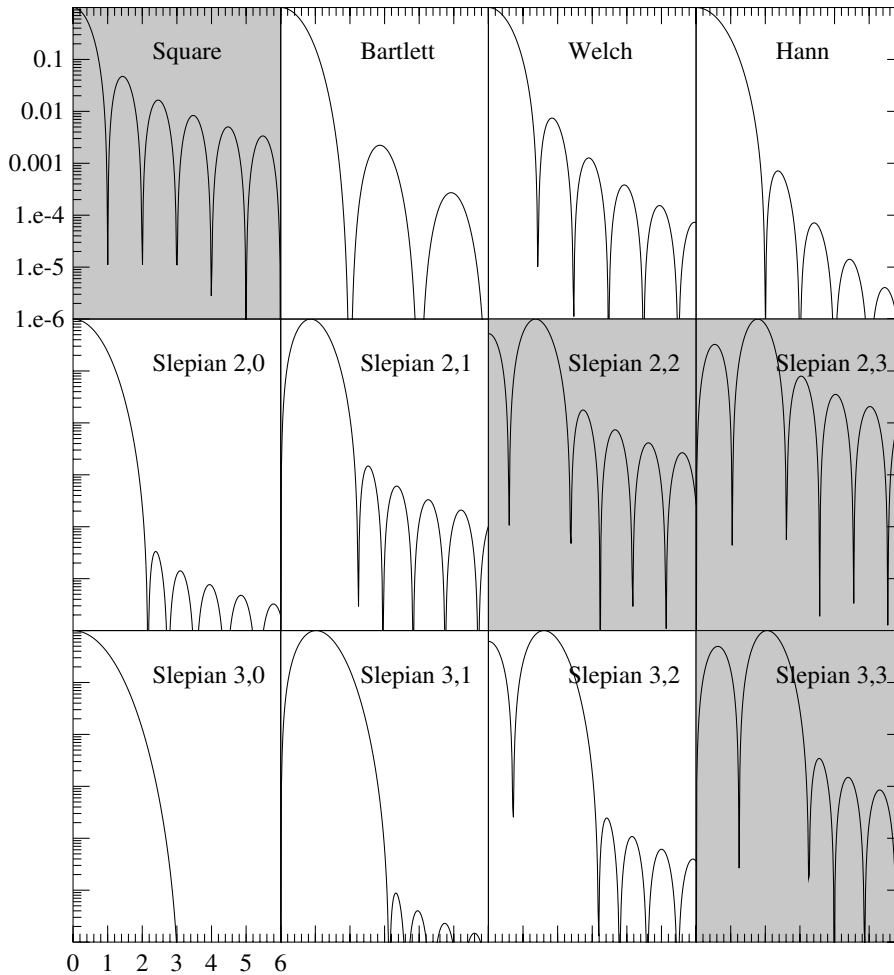


Figure 13.4.4. Leakage function $W(s)$ for various window functions. The top row is essentially the same as Figure 13.4.2, but squared (to get power) and plotted logarithmically. The second and third rows are examples of Slepian functions, identified by j_{res}, k values. Small k values have exceedingly small leakage for $|s| > j_{\text{res}}$; but as k increases, so does the leakage. Shaded functions have unacceptably large leakage and are not recommended.

```

MatDoub &dps;
Slepwindow(Int kkt, MatDoub &dpss) : k(kkt), dps(dpss) {}
Doub operator() (Int j, Int n) {return dps[k][j];}
};

void adddataseg(VecDoub_I &data) {
    Process a data segment of length  $2M$  using  $k_T$  tapers.
    Int k;
    if (data.size() != m2) throw("wrong size data segment");
    for (k=0; k<kT; k++) {
        Slepwindow window(k,dpss);
        Spectreg::adddataseg(data,window);
    }
}
};

```

We discuss the body of `filltable`, where the Slepian functions are actually computed, below.

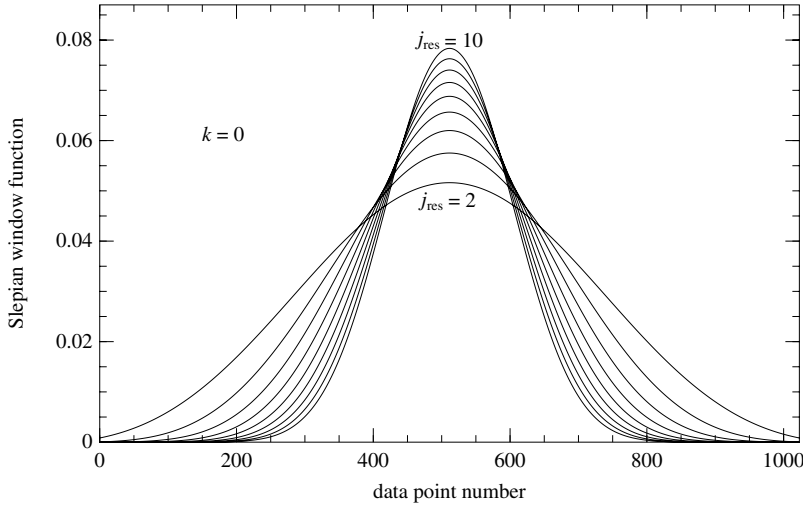


Figure 13.4.5. Slepian taper (window) functions for $k = 0$ (smallest eigenvalue) and $j_{\text{res}} = 2, 3, \dots, 10$, with $N = 1024$. Any of these, used by itself, is a good choice for the overlapping data segment method; see text.

First, a few words about the use and misuse of multitaper methods.

The Slepian multitaper method is fundamentally about low leakage. The fact that it can reduce the variance a bit by taking $k_T > 1$ is only a secondary consideration, because there are better ways to achieve the latter goal, for example by overlapping data segments. It follows that you should never need to take j_{res} or k_T very large, greater than 10, say. Your logical path for choosing parameters should be something like this: Leakage suppression of the Slepian functions is so amazingly good that you can get to any plausible desired level for the first few eigenvectors with modest j_{res} . Find that value, and the largest acceptable value for k_T . The frequency resolution is now j_{res} , measured in bins. You now pick M to get the physical frequency resolution that you actually need,

$$f_{\text{res}} = \frac{j_{\text{res}}}{2M\Delta} \quad (13.4.20)$$

(compare equation 13.4.6). Don't be too greedy, or you will produce an unacceptably large variance. Now, if you have N_{tot} data points, you process $N_{\text{tot}}/(2M)$ separate data segments using `adddataseg`.

It would be misguided to increase j_{res} to a large value just to increase k_T for the purpose of variance reduction. The reason is that, for a fixed desired physical frequency resolution, you will need to increase M in proportion, and thus decrease your number of separate data segments, also in proportion. You thus gain nothing in variance reduction, and (potentially) lose greatly in leakage.

If squeezing down the variance by the last little bit is important, then you might consider using only the first Slepian function for a given j_{res} , and then using overlapping data segments. You can code this using `Spectolap` and `Slepian` as models. As j_{res} increases, the optimal spacing of overlapped segments decreases, as you can intuit from the narrowing central peaks in Figure 13.4.5. A spacing of $0.7N/\sqrt{j_{\text{res}} + 0.3}$, that is, overlap of $N - 0.7N/\sqrt{j_{\text{res}} + 0.3}$, should be about right.

13.4.4 Computation of the Slepian Functions

We want to find the first few eigenvectors and eigenvalues of the tridiagonal matrix, equations (13.4.18) and (13.4.19). For $N \gg 1$ (always our situation), the eigenvalues are well separated and approximately a function of j_{res} only. A good starting approximation for the

smallest eigenvalue λ_0 is

$$\lambda_0 \approx 1.5692j_{\text{res}} - 0.10859 - 0.068762/j_{\text{res}}, \quad j_{\text{res}} \geq 1 \quad (13.4.21)$$

and a similar approximation for the spacing of the first two eigenvalues is

$$\lambda_1 - \lambda_0 \approx 3.1387j_{\text{res}} - 0.47276 - 0.20273/j_{\text{res}}, \quad j_{\text{res}} \geq 1 \quad (13.4.22)$$

With these hints, a workable strategy is to find the eigenvalues as roots of the characteristic polynomial, using Newton's method. As starting guesses, we use equations (13.4.21) and (13.4.22), and subsequently linear interpolation on λ_{k-1} and λ_k , to estimate λ_{k+1} . There exists a straightforward recurrence relation that evaluates the characteristic polynomial of a tridiagonal system and its first derivative simultaneously (see [6]), and more than three or four iterations are seldom required. Once the eigenvalue is in hand, the eigenvector is obtained by setting one component arbitrarily, solving the tridiagonal system for the other components, and then renormalizing the solution. The code uses an algebraically equivalent form for equation (13.4.18) that is less susceptible to roundoff error.

spectrum.h

```
void Slepian::filltable() {
    Calculate Slepian functions and store in table.
    const Doub EPS = 1.e-10, PI = 4.*atan(1.);
    Doub xx,xnew,xold,sw,ppp,ddd,sum,bet,ssub,ssup,*u;
    Int i,j,k,nl;
    VecDoub dg(m2),dgg(m2),gam(m2),sup(m2-1),sub(m2-1);
    sw = 2.*SQR(sin(jres*PI/m2));
    dg[0] = 0.25*(2*m2+sw*SQR(m2-1.))-1.;           Set up the tridiagonal matrix.
    for (i=1;i<m2;i++) {
        dg[i] = 0.25*(sw*SQR(m2-1.-2*i)+(2*(m2-i)-1.)*(2*i+1.));
        sub[i-1] = sup[i-1] = -i*(Doub)(m2-i)/2.;
    }
    xx = -0.10859 - 0.068762/jres + 1.5692*jres;      Eigenvalue first guess
    xold = xx + 0.47276 + 0.20273/jres - 3.1387*jres;
    for (k=0; k<kt; k++) {
        u = &dpss[k][0];                             Loop over number of desired eigenvalues.
        for (i=0;i<20;i++) {                             Point output vector into table.
            Loop over iterations of Newton's method.
            pp = 1.;
            p = dg[0] - xx;
            dd = 0.;
            d = -1.;
            for (j=1; j<m2; j++) {
                Recurrence evaluates polynomial and deriva-
                tive.
                ppp = pp; pp = p;
                ddd = dd; dd = d;
                p = pp*(dg[j]-xx) - ppp*SQR(sup[j-1]);
                d = -pp + dd*(dg[j]-xx) - ddd*SQR(sup[j-1]);
                if (abs(p)>1.e30) renorm(-100);
                else if (abs(p)<1.e-30) renorm(100);
            }
            xnew = xx - p/d;                             Newton's method.
            if (abs(xx-xnew) < EPS*abs(xnew)) break;
            xx = xnew;
        }
        xx = xnew - (xold - xnew);
        xold = xnew;
        for (i=0;i<m2;i++) dgg[i] = dg[i] - xnew;        Subtract eigenvalue from matrix
        nl = m2/3;                                       diagonal. Then, set one com-
        dgg[nl] = 1.;                                   ponent (saving current val-
        ssub = sup[nl]; ssub = sub[nl-1];               ues).
        u[0] = sup[nl] = sub[nl-1] = 0.;
        bet = dgg[0];
        for (i=1; i<m2; i++) {
            Begin tridiagonal solution.
            gam[i] = sup[i-1]/bet;
            bet = dgg[i] - sub[i-1]*gam[i];
            u[i] = ((i==nl? 1. : 0.) - sub[i-1]*u[i-1])/bet;
        }
    }
}
```